

Department of Artificial Intelligence and Data Science

B.E Elective IV: INFORMATION RETRIEVAL

2025-26

Prepared By.-- Ms. Chaitrali S. Kardile

Savitribai Phule Pune University
Fourth Year of Artificial Intelligence and Data Science (2020 Course)
(With effect from Academic Year 2023-24)

Semester-VII

Course Code	Course Name	Teaching Scheme (Hours/Week)			Examination Scheme and Marks						Credit Scheme			
		Lecture	Practical	Tutorial	In-Sem	End-Sem	Term work	Practical	Oral/ Presentation	Total	Lecture	Practical	Tutorial	Total
417521	Machine Learning	03	-	-	30	70	-	-	-	100	03	-	-	03
417522	Data Modeling & Visualization	03	-	-	30	70	-	-	-	100	03	-	-	03
417523	Elective III	03	-	-	30	70	-	-	-	100	03	-	-	03
417524	Elective IV	03	-	-	30	70	-	-	-	100	03	-	-	03
417525	Computer Laboratory I	-	04	-	-	-	50	25	-	75	-	02	-	02
417526	Computer Laboratory II	-	04	-	-	-	50	25	-	75	-	02	-	02
417527	Project Stage I	-	04	-	-	-	50	-	50	100	-	02	-	02
417528	MOOC	-	-	02	-	-	50	-	-	50	-	-	02	02
	Total	12	12	02	120	280	200	50	50	700	12	06	02	20
417529	Audit Course 7													
Total Credits											12	06	02	20

Elective III:

417523(A) Quantum Artificial Intelligence
417523(B) Industrial Internet of Things
417523(C) Enterprise Architecture and Components
417523(D) Bioinformatics

Computer Laboratory I:

It is based on two compulsory subjects:

- Machine Learning
- Data Modeling & Visualization

Elective IV:

417524(A) GPU Programming and Architecture
417524(B) Information Retrieval
417524(C) UI/UX Design
417524(D) Optimization Algorithms

Computer Laboratory II:

It is based on two Elective subjects:

- Elective III
- Elective IV

Audit Course 7:

AC7 – I Block Chain
AC7 – II Entrepreneurship Development
AC7 – III Botnet of Things
AC7 – IV Foreign Language
AC7 – V MOOC-Learn New Skills

Course Objectives

- To understand the basics of Information Retrieval
- To understand the concepts of Indexing & Query Processing for Information Retrieval
- To provide comprehensive details about various Evaluation methods
- To understand the different methods of Text Classification and Clustering
- To understand various search engine system operations and web structures
- To understand various applications of Information Retrieval

After completion of the course, learners should be able to

CO1: Understand the concept of Information Retrieval

CO2: Use an indexing approach for retrieval of documents

CO3: Evaluate and analyze the retrieved information

CO4: Apply appropriate method of Text Classification and Clustering

CO5: Design and implement innovative features in search engines

CO6: Analyze different real-life application of Information Retrieval

SYLLABUS

Introduction to information retrieval, Major challenges in IR, Features of an IR system, components of an IR model, IR system block diagram, Boolean retrieval, Information versus Data Retrieval, Text categorization, IR processes and fields, Vector Model, Probabilistic Model and Latent Semantic Indexing Model.

#Exemplar/Case Studies Visualization and Information Retrieval Techniques for Network Intrusion Detection

What is Information Retrieval?

- Definition: Information Retrieval (IR) is the science of searching for information within documents, searching for documents themselves, searching for metadata about documents, or searching within databases.
- Helping users find relevant information from a large collection of resources, often unstructured text.
- It's not just about matching keywords, but about understanding user intent and context to provide the most useful results.
- From web search engines (Google, Bing) to email search, enterprise search, and specialized databases, IR is everywhere.
- In today's data-rich world, efficient and effective information access is crucial for research, business, and daily life.

Detective Case Study: The Bengaluru Cyber Heist

Imagine a high-tech company called "FinTechGenius" in Bengaluru just got robbed by hackers, not with guns, but by stealing money digitally. The police have absolutely no idea who did it or how. Their only hope? A *massive* mountain of digital information they managed to grab from the company:

- Thousands of emails, chat messages (like WhatsApp and Slack) from *all* employees.
- Millions of tiny records from their computer servers, showing who logged in, when, and what they did.
- News articles and social media posts about the company.
- Even some secret messages from a shady online hacker group.

Your job, as super-smart AI & Data Science detectives, is to find the hidden clues in this mountain of data to catch the thieves.

1. "Data Tsunami"

The Problem: Imagine if the police handed you *physical printouts* of all those emails (thousands!) and server records (millions!). Could you read them all in a day? A week? A month? No way! You'd drown in paper. This is what we mean by "Information Overload" or the "Data Tsunami." There's so much data, you can't possibly look at it all manually.

2. The "Tricky Language"

The Problem:

- Same Word, Different Meanings: If you search for "bug," are you looking for a software problem or a creepy-crawly insect? If a hacker talks about "the cell," do they mean a phone or a prison?
- Different Words, Same Meaning: The hackers might not say "stole money." They might say "transferred funds," "moved assets," or even use a code word like "package delivered." If you only search for "stole money," you'll miss crucial clues.

3. Find the Real Clues

The Problem: If you just search for "security," you'll get every email about the office alarm system or password resets. Most of that isn't about the actual heist. You need to find the "relevant" clues – the ones that directly help solve the crime. An email mentioning "unusual login" might be way more important than 100 emails about "lunch plans."

4. Need for Speed

The Problem: The criminals are probably long gone. Every minute you spend manually searching is a minute they get further away. You need answers NOW.

1. First Scan (Casting a Wide Net): You'd tell the IR system: "Show me *anything* that mentions 'unauthorized access,' 'breach,' or 'suspicious activity' across all emails, chats, and logs." (This is for High Recall – making sure you don't miss *any* potentially relevant clue.) You'll get a lot of stuff, but you won't miss the key items.
2. Focusing In (Zeroing In): Then you'd refine: "Okay, from those suspicious messages, only show me the ones *between IT staff and senior managers* that happened *right before the heist date*." (This is for High Precision – getting rid of the noise and focusing on the most accurate results.)
3. Connecting the Dots (Ranking & Link Analysis): The IR system helps you connect an employee's chat message about "that new access code" to a server log showing an unusual login using a similar code. It links people, IP addresses, and actions across all the different piles of data. It even helps you see events in the right order (timeline).
4. Catching the Clues (Robustness): The system might even help you spot fake messages or "red herrings" left by the criminals trying to trick you.

Major Challenges

1. Ambiguity of Language
2. Relevance Assessment
3. Scale and Performance
4. Information Quality and Trust
5. Personalization vs. Privacy

Key Features

1. Efficiency and Scalability
2. High Recall
3. High Precision
4. Intelligent Ranking (Relevance Scoring)
5. User-Friendly Interface
6. Adaptability and Learning
7. Robustness and Anti-Spam Measures

Difference Between Data Retrieval and Information Retrieval

Data Retrieval (DR)

- Focus: Retrieving structured data from databases.
- Data Type: Data that is neatly organized into tables, rows, and columns, with predefined schemas (like an Excel spreadsheet or a typical SQL database).
- Queries: Precise, formal queries (like SQL queries). You ask for exact matches or values within defined ranges.
- Results: Exact matches. If the data exists and your query is correct, you get precisely what you asked for. No ambiguity.

Example: Finding a specific book by its ISBN number and exact title in a perfectly organized library where every book is in its precise spot.

Difference Between Data Retrieval and Information Retrieval

Information Retrieval (IR)

- Focus: Retrieving unstructured or semi-structured data (like text, images, audio, video).
- Data Type: Data that doesn't fit neatly into rigid tables. Its content is free-form and often ambiguous (e.g., documents, web pages, emails, social media posts).
- Queries: Often imprecise, natural language queries (keywords, phrases, questions). The goal is to find *relevant* information, not necessarily an exact match.
- Results: Ranked list of potentially relevant documents. There's an element of inference and estimation involved because "relevance" is subjective. You get the *best guesses* based on a scoring algorithm.

Example: Trying to find all books in a vast, sprawling library that talk *about* "the rise of artificial intelligence" (even if they don't use that exact phrase in the title), and getting the most helpful ones recommended to you first.

Difference Between Data Retrieval and Information Retrieval

Feature	Data Retrieval (DR)	Information Retrieval (IR)
Data Type	Structured data (e.g., databases, spreadsheets with predefined schemas).	Unstructured or semi-structured data (e.g., text documents, web pages, emails, images, audio, video).
Query Type	Precise, formal, unambiguous queries (e.g., SQL queries).	Often imprecise, natural language, or keyword-based queries.
Query Goal	To find exact matches based on specific criteria.	To find relevant information that satisfies a user's information need.
Results	Exact matches; either a record fits the criteria or it doesn't. Results are typically not ranked.	A ranked list of potentially relevant documents, ordered by their estimated relevance.
Ambiguity	Low tolerance for ambiguity. Queries must be exact.	High tolerance for ambiguity; deals with synonyms, polysemy, and context.
Underlying Task	Data management, record keeping, transactional processing.	Information discovery, knowledge extraction, understanding user intent.
Example	Finding all employees in the 'Sales' department hired after a specific date from a company database.	Searching Google for "best vegan restaurants in Pune" or finding relevant research papers on "quantum computing ethics."
Example	Finding a specific book by its exact ISBN in a perfectly organized library.	Finding books about a certain broad topic in a vast library, and getting the most helpful ones recommended first.

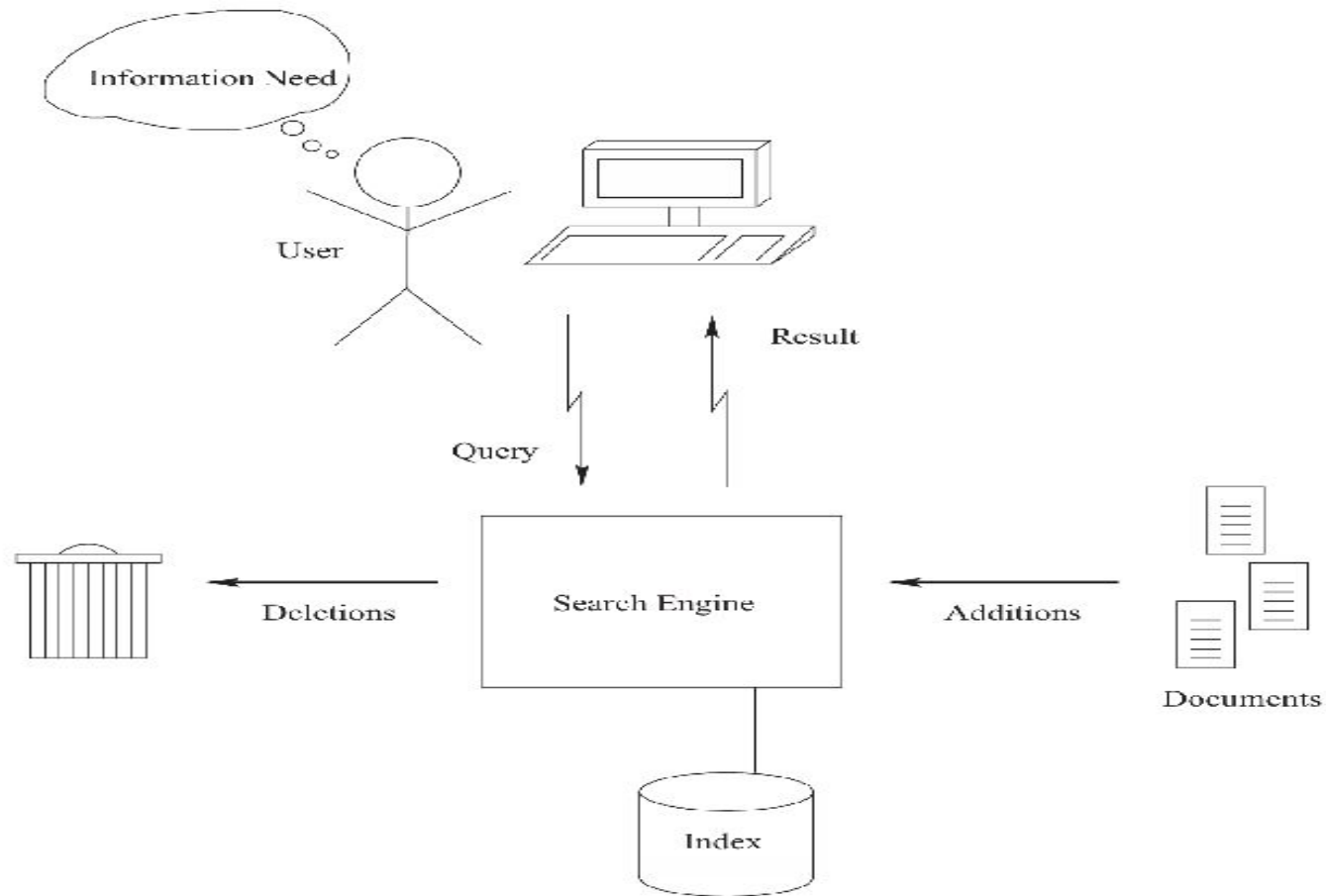
Components of an IR Model

- Document Representation: How documents are structured and stored (e.g., collection of terms).
- Query Representation: How user information needs are formally expressed.
- Matching Function (Scoring Function): Algorithm to determine the similarity between a query and a document.
- Ranking Algorithm: Orders the retrieved documents based on their scores.

Components of an IR Model

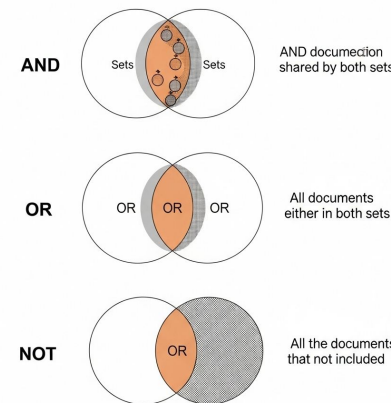
1. Document representation is about how we convert a raw document—be it a text file, an image, or a video—into a format that the system can process. For text, this often means representing it as a 'bag of words'.
2. query you type needs to be represented in a compatible format. This allows the system to compare apples to apples, or rather, document representations to query representations.
3. The matching function is the heart of the model. It's the mathematical magic that calculates how 'alike' a document is to your query. The higher the similarity score, the more relevant the document is considered.
4. the ranking algorithm takes these scores and presents the documents to you in a sorted list, with the most relevant ones at the top

IR system block diagram

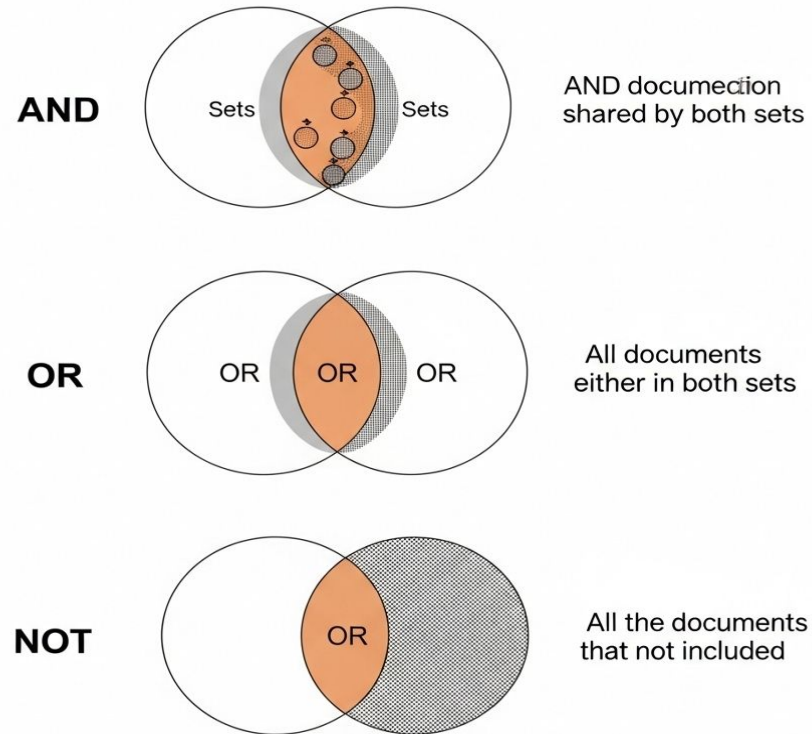


Boolean retrieval

- Boolean retrieval is one of the simplest and oldest models in IR. It's built on Boolean logic, where documents are retrieved based on the presence or absence of specific keywords combined with logical operators like AND, OR, and NOT.
- It's a strict filter. If you search for 'cat AND dog,' you'll only get documents that contain *both* words. If you search for 'cat OR dog,' you'll get documents with either or both.



Boolean retrieval



- Foundation: Based on Boolean algebra (AND, OR, NOT operators).
- Exact Match: Documents either fully match the query or don't. No partial relevance.
- Query Example: "forensic AND evidence AND NOT fingerprint"
- Advantages:
 - Precise when the user knows exactly what they want. Simple to implement.
- Disadvantages:
 - Lack of ranking: All retrieved documents are considered equally relevant.
 - Difficult for users to formulate complex Boolean queries.
 - Can lead to too many or too few results.
- Real-world use: Legal databases, medical information systems.

Text categorization

Text Categorization is a fundamental task in Information Retrieval that involves automatically assigning pre-defined categories or labels to text documents. Think of it as a sophisticated digital sorting system

Text categorization-What is Text Categorization

- Definition: The activity of classifying text documents into a set of pre-defined categories or classes.
- Key Idea: It's a form of supervised learning, meaning the system learns from a training set of documents where the correct categories are already known.
- Analogy: A detective sifting through hundreds of witness statements and automatically tagging them as "Alibi," "Eyewitness," or "Motive" based on their content, rather than manually sorting them.
- Core Goal: To automatically determine which category (or categories) a new, unseen document belongs to.

The Process of Text Categorization

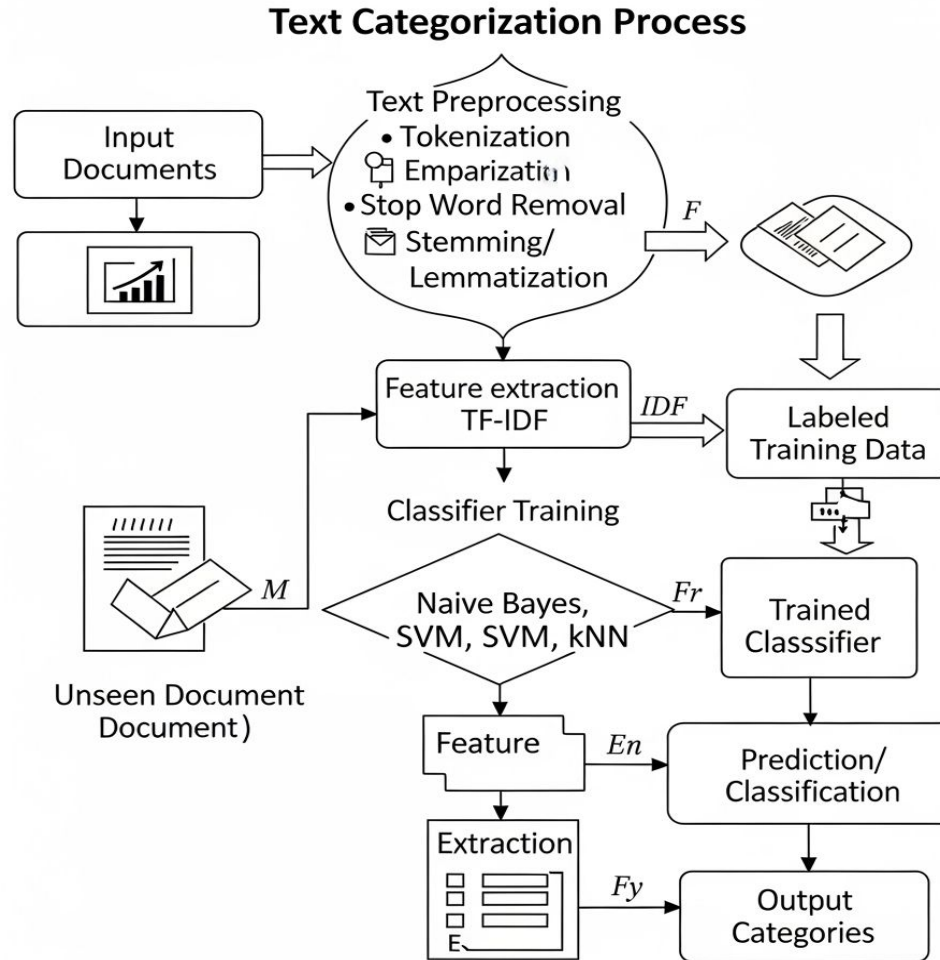
The process typically involves several key steps:

1. Feature Representation: Raw text cannot be directly fed into machine learning algorithms. Documents are transformed into numerical representations called feature vectors. Common techniques include:
 - Tokenization: Breaking text into individual words or phrases (tokens).
 - Stop Word Removal: Eliminating common, less informative words (e.g., "the", "is") that don't help differentiate categories.
 - Stemming/Lemmatization: Reducing words to their root form (e.g., "running", "ran", "runs" all become "run").
 - Term Weighting: Assigning numerical importance to terms, most commonly using TF-IDF (Term Frequency-Inverse Document Frequency), which highlights terms that are frequent in a document but rare across the entire collection.

The Process of Text Categorization

2. Training Data: A crucial component is a labeled dataset – a collection of documents where human experts have already assigned the correct categories. This dataset is used to train the classification model.
3. Classifier Algorithm Training: A machine learning algorithm is trained on the labeled data to learn the patterns and features associated with each category.
4. Prediction/Classification: Once trained, the classifier can take a new, unseen document, extract its features, and predict its most likely category (or categories).
5. Evaluation: The performance of the classifier is assessed using metrics such as Precision, Recall, and F1-score to determine how accurately it assigns categories

The Process of Text Categorization



1. Acquisition (Crawling/Harvesting):

- Purpose: Gathering raw digital documents from diverse sources (web, enterprise, social media, databases).
- Mechanism: Web crawlers, APIs.
- Example: A news aggregator constantly fetching articles from various online publishers.

2. Preprocessing / Text Operations:

- Purpose: Cleaning and normalizing raw text for efficient processing.
- Mechanism: Tokenization (breaking into words), Stop Word Removal (eliminating common words like 'the'), Stemming/Lemmatization (reducing words to root forms).
- Example: Converting "running shoes" and "ran" into a normalized form 'run'.

3. Indexing:

- Purpose: Creating fast lookup data structures (like an Inverted Index) to map terms to documents.
- Mechanism: Building a vocabulary of terms and associated document lists.
- Example: An index quickly pointing to all documents containing the term "forensic".

4. Storage:

- Purpose: Efficiently storing original documents and their corresponding indices.
- Mechanism: Optimized file systems, distributed storage.
- Example: Google's massive data centers storing billions of web pages and their indices.

5. Query Processing:

- Purpose: Transforming a user's natural language request into a formal query for the system.
- Mechanism: Query Analysis (understanding intent), Query Expansion (adding synonyms), Query Weighting, Spell Correction.
- Example: Expanding a query "AI ethics" to also search for "artificial intelligence morality".

6. Searching / Matching:

- Purpose: Identifying documents that are relevant to the processed query using the index and IR models.
- Mechanism: Boolean logic, Vector Space Model (cosine similarity), Probabilistic Models.
- Example: Finding documents where the query vector is geometrically "close" to document vectors.

7. Ranking:

- Purpose: Ordering retrieved documents by their estimated relevance, presenting the most useful first.
- Mechanism: Scoring functions (e.g., TF-IDF scores, PageRank for web pages).
- Example: Displaying the most authoritative and relevant websites at the top of search results.

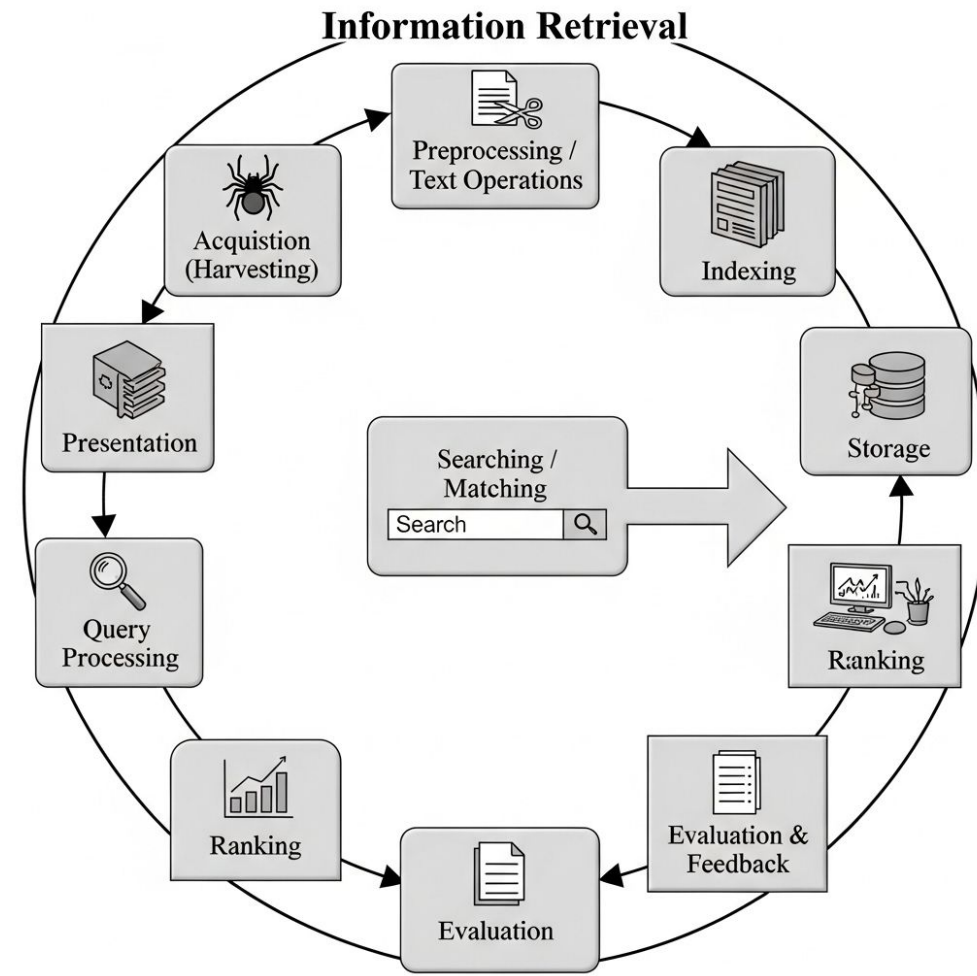
8. Presentation:

- Purpose: Displaying results to the user in an intuitive and actionable format.
- Mechanism: Ranked lists, snippets with highlighted terms, filtering options.
- Example: A search engine results page (SERP) showing title, URL, and a brief summary.

9. Evaluation & Feedback:

- Purpose: Measuring system effectiveness (Precision, Recall) and learning from user interactions.
- Mechanism: Explicit feedback (ratings), Implicit feedback (clicks, dwell time).
- Example: A system learning that documents clicked by users are more relevant, improving future rankings.

IR Processes



Documents and queries are represented as vectors in a high-dimensional space.

1. Term Weighting (TF-IDF):

- TF (Term Frequency): How often a term appears in a document. (Local importance)
 - *Example:* In a document about "detective work," the word "clue" might appear 10 times.
- IDF (Inverse Document Frequency): How unique a term is across the *entire collection*. (Global importance)
 - *Example:* "The" has a low IDF (appears everywhere), "fingerprint" has a high IDF (more specific).
- Combined TF-IDF: $wt_{t,d} = TF(t,d) \times IDF(t,D)$
 - A high TF-IDF means a term is important in *this* document and *rare* in the collection.

1. Similarity Measure: Cosine Similarity:

- Calculates the cosine of the angle between two vectors.
- Formula: $sim(Q,D) = \frac{\sum_{i=1}^N (w_{Q,i} \times w_{D,i})}{\sqrt{\sum_{i=1}^N w_{Q,i}^2} \times \sqrt{\sum_{i=1}^N w_{D,i}^2}}$
 - Where $w_{Q,i}$ is the weight of term i in query Q , and $w_{D,i}$ is the weight of term i in document D .
- Interpretation:
 - Cosine value close to 1: Vectors are very similar (small angle, highly relevant).
 - Cosine value close to 0: Vectors are dissimilar (large angle, less relevant).

1. Ranking: Documents are ranked in descending order of their cosine similarity scores with the query.

Vector Space Model

Document - “The quick brown fox jumps over the lazy dog. The fox is quick.”

Term	TF (Doc 1)	DF (Collection N=3)	IDF ($\log(N/DF)$)	TF-IDF (Doc 1)
"quick"	2	2	0.176	0.352
"fox"	2	1	0.477	0.954
"the"	2	2	0.176	0.352
"car"	0	1	0.477	0

1. **Web Search Engines:** The most prominent application (Google, Bing).
2. **Enterprise Search:** Searching internal documents within organizations.
3. **Digital Libraries and Archives:** Managing and retrieving academic papers, books, and historical records.
4. **E-commerce Search:** Product search on online retail platforms.
5. **Text Mining and Analytics:** Extracting patterns and insights from large text collections (e.g., sentiment analysis, topic modeling).
6. **Question Answering Systems:** Directly answering user questions from a knowledge base or document collection.
7. **Recommender Systems:** Suggesting items (products, movies, news) based on user preferences and content characteristics.
8. **Information Filtering:** Delivering relevant information streams (e.g., news feeds, email filters) to users based on their profiles.
9. **Network Intrusion Detection:** Analyzing log files to detect anomalies or malicious activities.
10. **Legal Tech:** Automating review and discovery of legal documents.
11. **Medical Informatics:** Retrieving relevant medical literature or patient data.

- The Probabilistic Model stands as a theoretically rigorous approach in Information Retrieval, fundamentally differing from models like the Boolean or Vector Space models by explicitly attempting to estimate the probability that a document is relevant to a given query. It moves beyond simple matching or similarity scores to quantify the *likelihood* of relevance.

1. Probability of Relevance ($P(R | D, Q)$)

At its heart, the probabilistic model's objective is to rank documents based on the decreasing order of their estimated probability of relevance to a user's query.

- Notation: This is formally expressed as $P(R | D, Q)$, which reads as "the probability of relevance (R) given a specific document (D) and a specific query (Q)".

2. The Probabilistic Ranking Principle (PRP)

The Probabilistic Ranking Principle (PRP) is the cornerstone of this approach. It states that an optimal retrieval system should rank documents in decreasing order of their probability of relevance to the user's query.

- **Optimality:** If the system can accurately estimate these probabilities, then presenting documents from highest probability to lowest will yield the most effective search results for the user.
- **Decision Boundary:** Implicitly, the PRP suggests that there's an optimal threshold: documents whose probability of relevance exceeds this threshold should be retrieved, and others should not.

3. How It Works (Intuitive Workflow)

1. Learning Characteristics: The model aims to learn the characteristics (e.g., specific terms, their frequencies, or patterns) that distinguish relevant documents (R) from non-relevant documents (R^-) with respect to a given query.
2. Probability Estimation: Using this learned information, when a new, unseen document comes in, the model estimates the probability that it belongs to the 'relevant' class for the query.
3. Ranking: Documents are then ordered based on these calculated probabilities.

To do this, probabilistic models often use Bayes' Theorem to calculate $P(R | D, Q)$. A common approach is to compare the odds of relevance: $P(R^- | D, Q)P(R | D, Q) = P(D | R^-, Q)P(R^- | Q)P(D | R, Q)P(R | Q)$

Since $P(R | Q)$ and $P(R^- | Q)$ are constant for a given query, the ranking is often based on the Retrieval Status Value (RSV), which is proportional to $P(D | R^-, Q)P(D | R, Q)$. This means we're comparing how likely the document is to be generated if it were relevant versus if it were non-relevant.

Advantages of Probabilistic Models

1. **Strong Theoretical Foundation:** Rooted in probability theory, offering a mathematically sound basis for ranking.
2. **Relevance Feedback Integration:** They naturally lend themselves to relevance feedback. As users provide feedback (marking documents as relevant or non-relevant), the system can update its probability estimates ($P(t | R)$ and $P(t | R^-)$), leading to improved future rankings. This is a very powerful feature.
3. **Intuitive Interpretation:** The notion of "probability of relevance" is very intuitive for users and system designers.

Disadvantages of Probabilistic Models

1. **Term Independence Assumption:** The assumption that terms appear independently is often unrealistic for natural language, where terms exhibit strong dependencies (e.g., "New" and "York"). This can limit the model's accuracy.
2. **Initial Probability Estimates:** Without prior knowledge or a pre-existing labeled training set of relevant/non-relevant documents, estimating the initial probabilities ($P(t | R)$ and $P(t | R^-)$) can be challenging. This is often referred to as the "cold start" problem.
3. **Binary Representation Limitation:** Simple binary models don't account for term frequency within a document, which can be an important signal of relevance. More advanced probabilistic models address this.

Latent Semantic Indexing (LSI) Model

- While Vector Space Model (VSM) works on literal term matching, LSI goes a step further to address the inherent challenges of natural language:
 - Synonymy: Different words having the same meaning (e.g., "car" and "automobile"). VSM struggles here.
 - Polysemy: The same word having multiple meanings (e.g., "bank" - river bank vs. financial bank). VSM can confuse these.
- Core Idea: LSI assumes that there's an underlying "latent semantic structure" in the data, where terms and documents are related not just by co-occurrence, but by abstract concepts.
- How it Works: Singular Value Decomposition (SVD):
 - Takes the term-document matrix (where rows are terms, columns are documents, and values are term weights like TF-IDF).
 - Applies Singular Value Decomposition (SVD) to this matrix. SVD is a mathematical technique that decomposes the original matrix into three simpler matrices.
 - Dimensionality Reduction: The key step is to *truncate* these matrices, keeping only the 'k' largest singular values. This effectively reduces the dimensionality of the term-document space, removing noise and highlighting the most important semantic relationships.
 - Concept Space: Documents and queries are then represented as vectors in this reduced-dimensional "latent semantic space" or "concept space."
- Retrieval: Queries are also transformed into this new concept space, and similarity (e.g., cosine similarity) is calculated between the query vector and document vectors in this reduced space.

Advantages

1. Handles Synonymy: Can retrieve documents relevant to a query even if they don't share any literal terms, because they share underlying concepts.
2. Handles Polysemy: By mapping terms to a conceptual space, it can disambiguate meanings to some extent.
3. Reduces Noise: Dimensionality reduction helps filter out less important variations and noise in the data.

Disadvantages

1. Computational Cost: SVD is computationally expensive for very large document collections, making real-time updates challenging.
2. Interpretability: The 'k' dimensions in the concept space are abstract and difficult for humans to interpret directly.
3. Determining 'k': Choosing the optimal number of dimensions ('k') is crucial and often requires experimentation.

THANK YOU

